

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

KHOA CÔNG NGHỆ THÔNG TIN



BÁO CÁO BÀI TẬP LỚN

CÁC HỆ THỐNG PHÂN TÁN

Đề tài: Phát triển hệ thống chat ngang hàng P2P

Giảng viên: Kim Ngọc Bách

Nhóm bài tập: 06

Lớp: 02

Thành viên: Lê Minh Châu – B22DCCN104

Nguyễn Tuấn Anh – B22DCCN032

Nguyễn Khánh Duy – B22DCCN152

Hà Nội - 2026

Mục lục

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI	4
1.1. Bối cảnh đề tài	4
1.2. Mục tiêu đề tài.....	4
1.3. Phạm vi đề tài.....	5
CHƯƠNG 2: CƠ SỞ LÝ THUYẾT	6
2.1. Tổng quan về mạng Peer-to-Peer (P2P).....	6
2.2. So sánh mô hình Client-Server và P2P	7
2.3. Cơ chế Peer Discovery.....	7
2.4 Bootstrap và tracker trong P2P	8
CHƯƠNG 3: PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG	9
3.1. Phân tích yêu cầu hệ thống.....	9
3.2. Kiến trúc tổng thể hệ thống.....	10
3.3. Thiết kế các thành phần chính	12
3.4. Giao thức và định dạng thông điệp.....	13
3.5. Cơ chế đồng bộ dữ liệu.....	14
3.6. Cơ chế chịu lỗi	14
CHƯƠNG 4: XÂY DỰNG VÀ CÀI ĐẶT HỆ THỐNG	16
4.1. Công nghệ sử dụng.....	16
4.2. Môi trường phát triển	19
CHƯƠNG 5: THỬ NGHIỆM VÀ ĐÁNH GIÁ	23
5.1. Kịch bản thử nghiệm	23
5.2. Kết quả thử nghiệm	25
5.3. Đánh giá hiệu năng.....	26
5.4. Đánh giá cơ chế xử lý lỗi	27
5.5. Hạn chế hệ thống.....	27
CHƯƠNG 6: KẾT QUẢ HỆ THỐNG	29
6.1. Kết quả xây dựng hệ thống	29
6.2. Giao diện và hoạt động của hệ thống	29

Danh mục hình ảnh

Hình 1 Sơ đồ kiến trúc tổng thể hệ thống.....	10
Hình 2 Sơ đồ luồng hoạt động của hệ thống	11
Hình 3 Giao diện chat.....	30
Hình 4 Giao diện người dùng (peer)	30
Hình 5 Giao diện chat 1-1.....	31
Hình 6 Giao diện tạo group chat.....	32
Hình 7 Giao diện group chat	33

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI

1.1. Bối cảnh đề tài

Trong các hệ thống chat truyền thống, máy chủ trung tâm thường đảm nhiệm toàn bộ quá trình tiếp nhận, lưu trữ, xử lý và định tuyến tin nhắn giữa các người dùng. Mô hình này giúp hệ thống dễ triển khai và quản lý, tuy nhiên lại tạo ra sự phụ thuộc lớn vào server trung tâm. Khi số lượng người dùng tăng cao, server có thể trở thành điểm nghẽn hiệu năng, đồng thời cũng là điểm lỗi duy nhất của hệ thống. Ngoài ra, toàn bộ dữ liệu giao tiếp đều đi qua server nên các vấn đề liên quan đến quyền riêng tư và bảo mật thông tin cũng trở nên đáng chú ý hơn.

Để giảm sự phụ thuộc vào hạ tầng tập trung, mô hình Peer-to-Peer (P2P) được sử dụng như một giải pháp cho các hệ thống phân tán hiện đại. Trong mô hình này, mỗi peer vừa đóng vai trò client để kết nối và trao đổi dữ liệu với mạng, vừa đóng vai trò server để tiếp nhận kết nối trực tiếp từ các peer khác. Nhờ đó, việc truyền dữ liệu có thể diễn ra trực tiếp giữa các node mà không cần thông qua server trung tâm trong quá trình chat.

Đề tài “Phát triển hệ thống chat ngang hàng P2P (Peer-to-Peer Chat System)” được xây dựng nhằm nghiên cứu và triển khai một hệ thống chat hoạt động theo mô hình phân tán. Trong hệ thống, mỗi peer có khả năng khám phá mạng thông qua bootstrap server hoặc tracker, đồng thời tự duy trì kết nối với các peer khác để gửi và nhận tin nhắn.

Theo yêu cầu của đề tài, hệ thống cần thể hiện được các đặc trưng cơ bản của hệ thống phân tán như: giao tiếp thông qua mạng, nhiều tiến trình hoạt động đồng thời, cơ chế peer discovery, quản lý trạng thái online/offline, xử lý lỗi kết nối, và cơ chế truyền tin đáng tin cậy giữa các peer.

1.2. Mục tiêu đề tài

Mục tiêu chính của đề tài là nghiên cứu và xây dựng hệ thống chat hoạt động theo mô hình Peer-to-Peer cho phép nhiều người dùng trao đổi dữ liệu trực tiếp thông qua mạng mà không phụ thuộc hoàn toàn vào server trung tâm.

Các mục tiêu cụ thể của đề tài bao gồm:

- Nghiên cứu mô hình mạng Peer-to-Peer và các cơ chế hoạt động của hệ thống phân tán.
- Xây dựng cơ chế peer discovery thông qua Bootstrap Server và Tracker.

- Thiết kế giao thức truyền thông giữa các peer bằng TCP Socket và JSON message.
- Xây dựng chức năng gửi và nhận tin nhắn trực tiếp giữa các peer.
- Hỗ trợ trò chuyện nhóm và cơ chế broadcast message.
- Xây dựng cơ chế heartbeat nhằm kiểm tra trạng thái hoạt động của peer.
- Lưu trữ lịch sử tin nhắn cục bộ bằng SQLite.
- Triển khai giao diện web realtime sử dụng FastAPI và WebSocket.
- Nghiên cứu cơ chế mã hóa dữ liệu nhằm tăng tính bảo mật cho hệ thống.
- Đánh giá khả năng hoạt động và mở rộng của hệ thống trong môi trường phân tán.

Thông qua đề tài, nhóm thực hiện hướng tới việc xây dựng một hệ thống chat P2P prototype có khả năng hoạt động ổn định, hỗ trợ realtime communication và phản ánh được các đặc trưng cơ bản của mạng ngang hàng.

1.3. Phạm vi đề tài

Đề tài tập trung nghiên cứu và xây dựng hệ thống chat ngang hàng P2P nhằm mô phỏng các đặc trưng cơ bản của hệ thống phân tán. Nội dung chính của đề tài bao gồm cơ chế peer discovery thông qua Bootstrap Server, giao tiếp giữa các peer bằng TCP Socket, gửi và nhận tin nhắn trực tiếp, chat nhóm, heartbeat, ACK/retry và lưu trữ lịch sử chat bằng SQLite. Ngoài ra, hệ thống cũng hỗ trợ giao diện realtime sử dụng WebSocket và cơ chế mã hóa dữ liệu cơ bản.

Về phạm vi không gian, hệ thống được triển khai và thử nghiệm trong môi trường local và mạng LAN giữa nhiều máy tính khác nhau nhằm đánh giá khả năng kết nối và hoạt động của các peer trong mạng P2P.

Về phạm vi thời gian, quá trình nghiên cứu, xây dựng và thử nghiệm hệ thống được thực hiện trong thời gian thực hiện đồ án. Các kết quả và đánh giá trong báo cáo được dựa trên phiên bản hiện tại của hệ thống tại thời điểm hoàn thành đề tài.

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT

2.1. Tổng quan về mạng Peer-to-Peer (P2P)

2.1.1. Khái niệm mạng Peer-to-Peer

Mạng Peer-to-Peer (P2P) là mô hình mạng trong đó các thiết bị tham gia mạng, gọi là peer, có vai trò ngang hàng với nhau. Mỗi peer vừa có thể đóng vai trò client để gửi yêu cầu, vừa có thể đóng vai trò server để cung cấp dịch vụ hoặc tài nguyên cho các peer khác.

Khác với mô hình Client-Server truyền thống, mạng P2P không phụ thuộc hoàn toàn vào một máy chủ trung tâm để xử lý toàn bộ dữ liệu và điều phối giao tiếp. Các peer có thể kết nối trực tiếp với nhau để trao đổi dữ liệu, chia sẻ tài nguyên hoặc truyền tải thông điệp.

Trong hệ thống chat P2P, mỗi người dùng là một peer trong mạng. Khi một peer gửi tin nhắn, dữ liệu có thể được truyền trực tiếp đến peer đích hoặc được chuyển tiếp thông qua các peer khác trong mạng.

2.1.2. Đặc điểm của mạng P2P

Mạng P2P có các đặc điểm chính như sau:

- Các node trong mạng hoạt động ngang hàng.
- Không phụ thuộc hoàn toàn vào server trung tâm.
- Tài nguyên được phân tán trên nhiều peer.
- Có khả năng mở rộng tốt khi số lượng peer tăng.
- Có khả năng chịu lỗi cao do không tồn tại một điểm lỗi tập trung.

Tuy nhiên, mạng P2P cũng tồn tại một số hạn chế như:

- Khó quản lý hệ thống khi số lượng peer lớn.
- Khó đảm bảo tính bảo mật và xác thực.
- Việc tìm kiếm peer và định tuyến dữ liệu phức tạp hơn mô hình tập trung.

2.1.3. Ứng dụng của mạng P2P

Mạng P2P được ứng dụng trong nhiều lĩnh vực khác nhau như: hệ thống chia sẻ tệp tin (BitTorrent), Blockchain, hệ thống gọi thoại và video trực tuyến, hệ thống chat thời gian thực và chia sẻ dữ liệu phân tán.

Hiện nay, mô hình P2P được sử dụng rộng rãi nhờ khả năng giảm tải cho server trung tâm và tăng tính linh hoạt cho hệ thống.

2.2. So sánh mô hình Client-Server và P2P

Mô hình Client-Server là kiến trúc phổ biến trong các hệ thống mạng truyền thống. Trong mô hình này, toàn bộ client phải gửi yêu cầu đến server trung tâm để xử lý dữ liệu.

Ngược lại, mô hình P2P cho phép các peer giao tiếp trực tiếp với nhau mà không cần phụ thuộc hoàn toàn vào server trung tâm.

Tiêu chí	Client-Server	Peer-to-Peer
Kiến trúc	Tập trung	Phân tán
Server trung tâm	Bắt buộc	Không bắt buộc
Khả năng mở rộng	Trung bình	Cao
Điểm lỗi tập trung	Có	Hạn chế
Chi phí server	Cao	Thấp hơn
Quản lý hệ thống	Dễ	Khó hơn

2.3. Cơ chế Peer Discovery

Peer Discovery là cơ chế giúp các peer tìm kiếm và kết nối với nhau trong mạng P2P. Trong hệ thống P2P, khi một peer mới tham gia mạng, peer đó cần biết địa chỉ của các peer khác để thiết lập kết nối.

Một số phương pháp Peer Discovery phổ biến:

2.5.1. Bootstrap Server

Bootstrap server lưu danh sách các peer đang hoạt động trong mạng. Khi một peer mới tham gia:

1. Peer gửi yêu cầu đến bootstrap server.
2. Bootstrap server trả về danh sách peer.
3. Peer thực hiện kết nối trực tiếp với các peer khác.

Đây là phương pháp được sử dụng trong đề tài do dễ triển khai và phù hợp với hệ thống quy mô nhỏ và trung bình.

2.5.2. Gossip Protocol

Gossip Protocol là cơ chế các peer trao đổi thông tin ngẫu nhiên với nhau để cập nhật trạng thái mạng.

Ưu điểm:

- Tính phân tán cao.
- Không phụ thuộc server trung tâm.

Nhược điểm:

- Có thể phát sinh nhiều lưu lượng mạng.

2.5.3. Distributed Hash Table (DHT)

DHT là cơ chế quản lý dữ liệu phân tán thường dùng trong Structured P2P. Mỗi peer lưu trữ một phần dữ liệu định tuyến giúp tìm kiếm peer nhanh hơn.

DHT thường được sử dụng trong các hệ thống P2P quy mô lớn như BitTorrent.

2.4 Bootstrap và tracker trong P2P

Bootstrap Server đóng vai trò là điểm khởi tạo kết nối trong mạng P2P. Khi một peer khởi động, peer sẽ gửi thông tin như Peer ID, địa chỉ IP và port tới Bootstrap Server. Server sẽ lưu thông tin này và trả về danh sách các peer đang hoạt động để peer mới có thể thiết lập kết nối trực tiếp với các node khác trong hệ thống.

Bên cạnh đó, Tracker có nhiệm vụ theo dõi trạng thái hoạt động của các peer, bao gồm cập nhật peer online/offline và hỗ trợ quản lý thông tin kết nối. Tracker giúp các peer mới nhanh chóng tìm được node phù hợp mà không cần broadcast trên toàn mạng.

Sau khi kết nối ban đầu được thiết lập thông qua Bootstrap Server và Tracker, việc gửi và nhận tin nhắn sẽ diễn ra trực tiếp giữa các peer. Điều này giúp giảm tải cho server trung tâm và đảm bảo tính phân tán của mô hình P2P.

Trong phạm vi đề tài, Bootstrap Server và Tracker được lựa chọn vì có cấu trúc đơn giản, dễ triển khai và phù hợp với hệ thống chat P2P quy mô nhỏ và trung bình phục vụ mục đích học tập và nghiên cứu.

CHƯƠNG 3: PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

3.1. Phân tích yêu cầu hệ thống

3.1.1. Mục tiêu hệ thống

Hệ thống chat ngang hàng P2P được xây dựng nhằm cho phép nhiều người dùng trao đổi tin nhắn trực tiếp thông qua mạng mà không phụ thuộc hoàn toàn vào server trung tâm.

Trong hệ thống này, mỗi peer vừa đóng vai trò client để gửi yêu cầu, vừa đóng vai trò server để nhận kết nối và xử lý tin nhắn từ các peer khác.

Hệ thống hướng đến các mục tiêu:

- Cho phép nhiều peer giao tiếp thời gian thực.
- Giảm tải cho server trung tâm.
- Tăng khả năng mở rộng hệ thống.
- Hỗ trợ peer discovery.
- Hỗ trợ chuyển tiếp tin nhắn giữa các peer.

3.1.2. Yêu cầu chức năng

Hệ thống cần đáp ứng các yêu cầu chức năng sau:

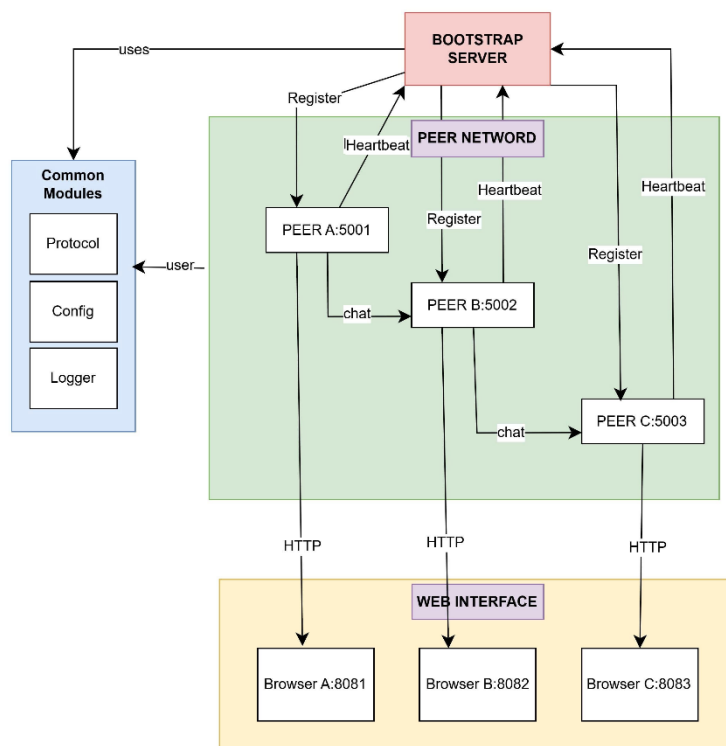
Yêu cầu	Mô tả
Kết nối vào mạng P2P	Peer có thể tham gia mạng thông qua bootstrap server. Bootstrap server cung cấp danh sách peer đang hoạt động.
Gửi và nhận tin nhắn	Peer có thể gửi tin nhắn đến peer khác. Peer có thể nhận và hiển thị tin nhắn theo thời gian thực.
Quản lý danh sách peer	Hiển thị danh sách peer online. Cập nhật danh sách khi peer tham gia hoặc rời mạng.
Chuyển tiếp tin nhắn	Trong trường hợp peer đích không kết nối trực tiếp, hệ thống hỗ trợ chuyển tiếp thông điệp qua peer trung gian.
Kiểm tra trạng thái kết nối	Hệ thống sử dụng heartbeat hoặc ping để kiểm tra peer còn hoạt động hay không.

3.1.3. Yêu cầu phi chức năng

Ngoài các chức năng chính, hệ thống cần đáp ứng một số yêu cầu phi chức năng như:

Yêu cầu	Mô tả
Tính ổn định	Hệ thống cần duy trì hoạt động khi một số peer rời mạng hoặc mất kết nối.
Khả năng mở rộng	Hệ thống cần hỗ trợ số lượng peer lớn hơn mà không làm giảm đáng kể hiệu năng.
Độ trễ thấp	Tin nhắn cần được truyền tải nhanh giữa các peer.
Tính linh hoạt	Các peer có thể tham gia hoặc rời mạng bất kỳ lúc nào.
Khả năng chịu lỗi	Hệ thống cần có cơ chế phát hiện và xử lý lỗi kết nối.

3.2. Kiến trúc tổng thể hệ thống



Hình 1 Sơ đồ kiến trúc tổng thể hệ thống

3.2.1. Mô hình kiến trúc

Hệ thống được xây dựng theo mô hình Hybrid Peer-to-Peer. Trong mô hình này, Bootstrap Server chỉ hỗ trợ peer discovery và không tham gia trực tiếp vào quá trình truyền tải dữ liệu giữa các peer.

Kiến trúc hệ thống gồm ba thành phần chính:

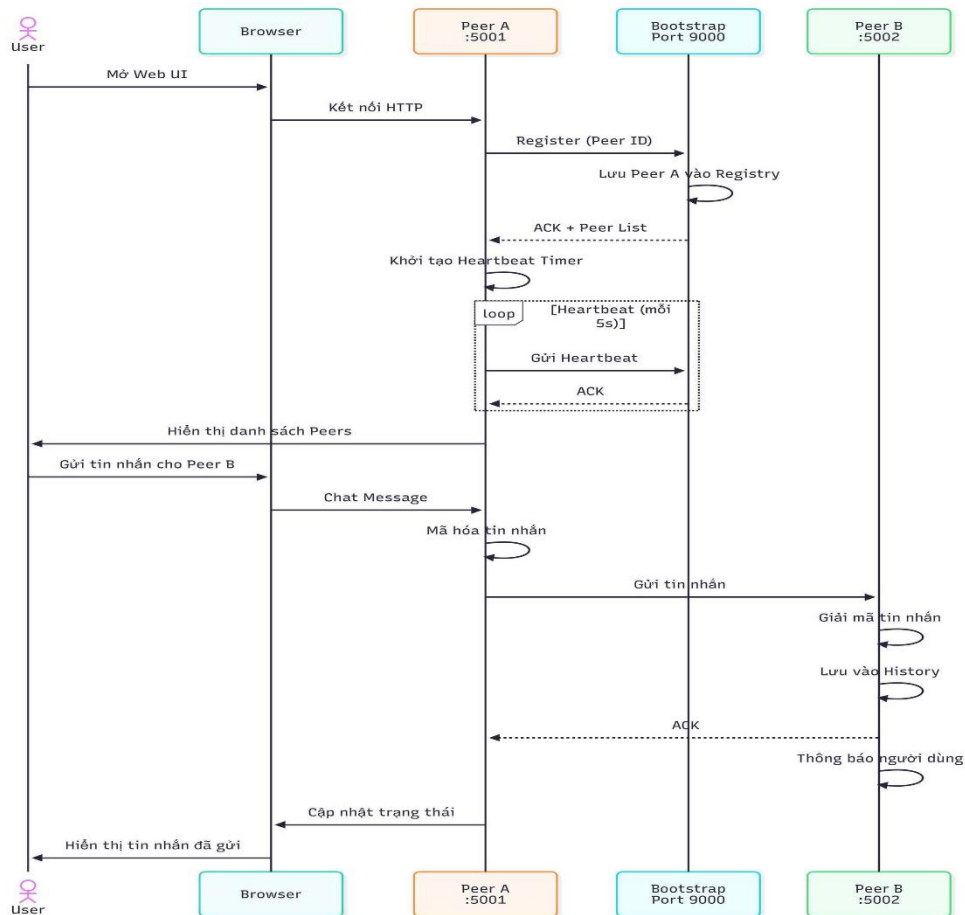
- Bootstrap Server
- Peer Node
- Web UI

Bootstrap Server chịu trách nhiệm quản lý danh sách peer đang hoạt động và cung cấp thông tin kết nối cho các peer mới tham gia mạng.

Peer Node là thành phần trung tâm của hệ thống. Mỗi peer vừa đóng vai trò server để nhận kết nối, vừa đóng vai trò client để gửi dữ liệu đến peer khác. Peer Node quản lý danh sách peer, xử lý heartbeat, đồng bộ dữ liệu và lưu trữ lịch sử tin nhắn.

Web UI được xây dựng bằng FastAPI và WebSocket nhằm cung cấp giao diện realtime cho người dùng.

3.2.2. Luồng hoạt động của hệ thống



Hình 2 Sơ đồ luồng hoạt động của hệ thống

Khi một peer khởi động, peer sẽ gửi yêu cầu đăng ký tới Bootstrap Server. Sau đó Bootstrap Server trả về danh sách các peer hiện có trong mạng.

Peer sẽ thực hiện kết nối TCP tới một số peer trong danh sách nhận được và bắt đầu trao đổi Peer List với các node khác. Sau khi kết nối được thiết lập, hệ thống khởi động cơ chế heartbeat để duy trì trạng thái kết nối và bắt đầu lắng nghe dữ liệu từ mạng.

Quá trình gửi và nhận tin nhắn diễn ra trực tiếp giữa các peer mà không cần thông qua Bootstrap Server.

3.3. Thiết kế các thành phần chính

Hệ thống được thiết kế và xây dựng dựa trên các thành phần chính sau:

Bootstrap Server: Đóng vai trò hỗ trợ peer discovery trong hệ thống P2P Chat. Khi một peer mới tham gia mạng, peer sẽ gửi thông tin gồm Peer ID, địa chỉ IP và port tới Bootstrap Server để đăng ký. Server sẽ lưu các thông tin này vào registry và trả về danh sách các peer đang hoạt động nhằm hỗ trợ thiết lập kết nối giữa các node trong mạng. Ngoài ra, Bootstrap Server còn sử dụng heartbeat timeout để loại bỏ các peer không còn hoạt động.

Peer Node: Là thành phần trung tâm của hệ thống, mỗi peer vừa đóng vai trò client vừa đóng vai trò server. Peer chịu trách nhiệm thiết lập kết nối TCP, gửi và nhận dữ liệu, mã hóa và giải mã thông điệp, đồng bộ lịch sử chat và quản lý trạng thái hoạt động của các peer khác trong mạng. Mỗi Peer Node duy trì các thành phần như PeerList, HistoryStore, HeartbeatManager và EventHub.

PeerList: Là thành phần quản lý danh sách các peer đang kết nối trong hệ thống. Mỗi bản ghi PeerInfo bao gồm các thông tin như peer_id, host, port, status, last_seen và writer. Để tránh race condition trong môi trường bất đồng bộ, PeerList sử dụng `asyncio.Lock()` khi cập nhật dữ liệu.

HistoryStore: Sử dụng SQLite để lưu trữ lịch sử tin nhắn cục bộ. Cơ sở dữ liệu gồm bảng messages để lưu thông tin tin nhắn và bảng sync_state để lưu trạng thái đồng bộ giữa các peer. Tin nhắn chỉ được lưu khi gửi thành công và nhận ACK hoặc khi nhận dữ liệu từ peer khác.

HeartbeatManager: Chịu trách nhiệm duy trì trạng thái hoạt động của peer trong mạng. Theo chu kỳ định sẵn, hệ thống sẽ gửi heartbeat tới Bootstrap Server và các peer đang kết nối nhằm kiểm tra trạng thái hoạt động. Nếu thời gian last_seen vượt quá heartbeat_timeout, peer sẽ bị đánh dấu offline và loại khỏi PeerList.

EventHub: Hoạt động theo cơ chế publish-subscribe nhằm hỗ trợ giao diện realtime. Khi có các sự kiện như `message_received`, `message_sent`, `peer_joined` hoặc `peer_left`, EventHub sẽ phát dữ liệu tới các subscriber thông qua queue bất đồng bộ để cập nhật giao diện người dùng theo thời gian thực.

Crypto Module: Chịu trách nhiệm mã hóa và giải mã dữ liệu trước khi truyền tải giữa các peer. Hệ thống sử dụng cơ chế mã hóa đối xứng với keystream được sinh từ SHA-256 dựa trên shared secret, nonce và counter. Dữ liệu plaintext được XOR với keystream và encode bằng base64-url trước khi gửi qua mạng. Tuy nhiên, cơ chế này chỉ mang tính minh họa và chưa đạt mức bảo mật như các chuẩn hiện đại như AES-GCM.

3.4. Giao thức và định dạng thông điệp

3.4.1. Giao thức truyền tải

Hệ thống sử dụng TCP stream kết hợp line-delimited JSON messages để truyền dữ liệu giữa các peer.

Mỗi message gồm các trường cơ bản:

- `Type`: Xác định loại thông điệp được gửi trong hệ thống.
- `from`: Lưu thông tin peer gửi message.
- `timestamp`: Ghi lại thời gian tạo hoặc gửi message.
- `message_id`: Định danh duy nhất cho mỗi message nhằm hỗ trợ ACK và deduplication.
- `version`: Xác định phiên bản giao thức truyền thông đang sử dụng.
- `payload`: Chứa nội dung dữ liệu chính của message như tin nhắn hoặc thông tin peer.

3.4.2. Các loại thông điệp

Hệ thống sử dụng nhiều loại thông điệp khác nhau như:

- `REGISTER`: Đăng ký peer mới với Bootstrap Server khi tham gia mạng.
- `GET_PEERS`: Yêu cầu danh sách các peer đang hoạt động trong hệ thống.
- `PEER_LIST`: Trả về danh sách các peer hiện có trong mạng.
- `PEER_HELLO`: Thực hiện handshake và xác nhận kết nối giữa các peer.
- `HEARTBEAT`: Kiểm tra trạng thái hoạt động và duy trì kết nối giữa các peer.
- `CHAT`: Gửi tin nhắn trực tiếp giữa hai peer.
- `ACK`: Xác nhận đã nhận message thành công.

- GROUP_CHAT: Gửi tin nhắn tới nhiều peer trong cùng nhóm chat.
- PEER_JOINED: Thông báo có peer mới tham gia mạng hoặc nhóm chat.
- PEER_LEFT: Thông báo peer đã rời mạng hoặc mất kết nối.

Khi thiết lập kết nối TCP mới, peer sẽ gửi thông điệp PEER_HELLO để trao đổi thông tin nhận dạng và địa chỉ kết nối.

3.4.4. Delivery Semantics

Hệ thống sử dụng cơ chế “at-least-once delivery” kết hợp ACK và retry.

Quy trình gửi tin:

1. Peer gửi message.
2. Chờ ACK từ peer nhận.
3. Nếu timeout sau 5 giây, hệ thống retry tối đa 3 lần.

Ngoài ra, hệ thống sử dụng message_id cùng TTL 300 giây để chống xử lý trùng lặp.

3.5. Cơ chế đồng bộ dữ liệu

Khi reconnect với peer khác, node sẽ gửi SYNC_REQUEST chứa thời điểm last_sync.

Peer nhận sẽ trả về các tin nhắn mới thông qua SYNC_RESPONSE nhằm đảm bảo lịch sử chat được cập nhật sau khi mất kết nối.

3.6. Cơ chế chịu lỗi

Hệ thống được thiết kế với cơ chế graceful degradation nhằm duy trì hoạt động ngay cả khi Bootstrap Server không khả dụng. Trong trường hợp này, các peer vẫn có thể sử dụng seed nodes đã biết trước để thiết lập kết nối ban đầu và tiếp tục duy trì giao tiếp trong mạng.

Ngoài ra, hệ thống còn hỗ trợ cơ chế PEER_EXCHANGE cho phép các peer trao đổi danh sách node với nhau. Nhờ đó, peer mới vẫn có thể discovery các peer khác mà không phụ thuộc hoàn toàn vào Bootstrap Server.

Để tăng độ tin cậy trong quá trình truyền tin, hệ thống sử dụng cơ chế retry và timeout detection. Khi không nhận được ACK trong khoảng thời gian quy định, message sẽ được gửi lại nhiều lần trước khi đánh dấu thất bại.

Hệ thống cũng triển khai heartbeat định kỳ nhằm kiểm tra trạng thái hoạt động của các peer. Nếu một peer không phản hồi heartbeat trong khoảng thời gian heartbeat_timeout, peer đó sẽ bị xem là offline và loại khỏi PeerList.

Ngoài ra, cơ chế deduplication được sử dụng để tránh xử lý trùng lặp message. Mỗi message được gán một message_id duy nhất và được lưu tạm trong cache TTL để hệ thống có thể phát hiện và bỏ qua các message đã xử lý trước đó.

CHƯƠNG 4: XÂY DỰNG VÀ CÀI ĐẶT HỆ THỐNG

4.1. Công nghệ sử dụng

Hệ thống P2P Chat được xây dựng bằng ngôn ngữ Python kết hợp với các thư viện hỗ trợ lập trình mạng bất đồng bộ, giao diện web realtime và lưu trữ dữ liệu cục bộ. Việc lựa chọn các công nghệ này nhằm đảm bảo hệ thống có khả năng mở rộng, dễ triển khai và phù hợp với mô hình phân tán của mạng Peer-to-Peer.

4.1.1. Ngôn ngữ lập trình

Ngôn ngữ chính được sử dụng là Python với phiên bản tương thích cùng các thư viện như typer, Click, fastapi, uvicorn, prompt-toolkit, rich, pyyaml.

Python được lựa chọn nhờ:

- cú pháp đơn giản, dễ phát triển.
- Hỗ trợ mạnh cho lập trình mạng và bất đồng bộ.
- Hệ sinh thái thư viện phong phú.
- Khả năng phát triển prototype nhanh.

4.1.2. Framework và thư viện chính

Hệ thống sử dụng nhiều framework và thư viện hỗ trợ cho các chức năng khác nhau.

Thư viện	Chức năng
FastAPI và Uvicorn	Xây dựng REST API và WebSocket server phục vụ giao diện web realtime. Uvicorn đóng vai trò ASGI server để chạy ứng dụng FastAPI.
Typer và Click	Xây dựng giao diện dòng lệnh (CLI) giúp người dùng khởi chạy và cấu hình peer một cách linh hoạt.
Prompt-toolkit	Hỗ trợ xây dựng giao diện CLI tương tác trực tiếp trên terminal, cho phép người dùng gửi lệnh và xem thông tin realtime.
Rich	Hiển thị logging và console output với định dạng trực quan hơn.

PyYAML	Hỗ trợ đọc và xử lý file cấu hình YAML cho hệ thống.
--------	--

4.1.3. Công nghệ mạng và giao tiếp

a. TCP Socket

Hệ thống sử dụng TCP Socket thông qua asyncio stream để xây dựng giao tiếp peer-to-peer.

TCP được lựa chọn do:

- Đảm bảo dữ liệu truyền đúng thứ tự.
- Hạn chế mất gói tin.
- Hỗ trợ kết nối hai chiều ổn định.

Dữ liệu giữa các peer được biểu diễn dưới dạng line-delimited JSON messages.

b. WebSocket

WebSocket được sử dụng trong FastAPI nhằm cung cấp khả năng cập nhật dữ liệu realtime giữa backend và giao diện web.

Các sự kiện như:

- nhận tin nhắn mới,
- cập nhật peer online,
- trạng thái gửi tin

được push trực tiếp tới frontend thông qua WebSocket.

4.1.4. Công nghệ lưu trữ

Hệ thống sử dụng SQLite thông qua module sqlite3 để lưu lịch sử tin nhắn cục bộ.

SQLite được lựa chọn nhờ:

- Nhẹ và dễ triển khai.
- Không cần server database riêng.
- Phù hợp với ứng dụng desktop hoặc peer local.

HistoryStore chịu trách nhiệm quản lý việc ghi và truy xuất dữ liệu từ SQLite.

4.1.5. Bảo mật dữ liệu

Hệ thống triển khai cơ chế mã hóa đối xứng đơn giản trong file `crypto.py`.

Keystream được sinh từ:

- shared secret
- nonce
- SHA-256

Sau đó dữ liệu plaintext được XOR với keystream trước khi truyền qua mạng.

4.1.6. Đồng bộ và bất đồng bộ

Toàn bộ hệ thống sử dụng thư viện `asyncio` để xử lý:

- I/O mạng,
- concurrency,
- background tasks,
- heartbeat,
- retry mechanism.

Việc sử dụng mô hình bất đồng bộ giúp hệ thống:

- xử lý nhiều kết nối đồng thời,
- giảm blocking I/O,
- tăng hiệu năng realtime.

4.1.7. Logging và Metrics

Hệ thống sử dụng module `metrics.py` để theo dõi:

- số lượng tin nhắn gửi/nhận,
- ACK timeout,
- trạng thái kết nối,
- message retry.

Thông tin logging được hiển thị thông qua Rich nhằm hỗ trợ debugging và giám sát hệ thống.

4.1.8. Công cụ kiểm thử

Hệ thống sử dụng pytest để thực hiện kiểm thử tự động.

Các test được cài đặt thông qua file requirements-dev.txt nhằm hỗ trợ:

- unit test,
- integration test,
- kiểm thử message flow,
- kiểm thử peer discovery.

4.2. Môi trường phát triển

4.2.1. Yêu cầu phần mềm

Hệ thống yêu cầu:

- Python 3.10 trở lên,
- môi trường ảo venv,
- các package được khai báo trong requirements.txt.

4.2.2. Cài đặt môi trường

Quá trình cài đặt bắt đầu bằng việc tạo virtual environment và cài đặt dependencies cần thiết.

Việc sử dụng virtual environment giúp tách biệt môi trường phát triển, tránh xung đột package, dễ dàng triển khai trên nhiều máy khác nhau.

4.2.3. Triển khai local

Trong môi trường local, hệ thống được chạy theo quy trình:

1. Khởi động Bootstrap Server.
2. Khởi động các Peer Node.
3. Kết nối giao diện Web UI.
4. Bắt đầu trao đổi tin nhắn.

4.2.4. Triển khai multi-machine

Để triển khai trên nhiều máy khác nhau, hệ thống sử dụng:

- địa chỉ IP quảng bá (`advertise_host`),
- mở firewall cho cổng peer và web,
- hoặc sử dụng ZeroTier trong trường hợp NAT/CGNAT.

Điều này cho phép các peer kết nối trực tiếp qua mạng LAN hoặc Internet.

4.2.5. Cấu hình hệ thống

Hệ thống hỗ trợ nhiều phương thức cấu hình nhằm tăng tính linh hoạt trong quá trình triển khai và vận hành. Người dùng có thể cấu hình hệ thống thông qua file JSON, file YAML hoặc sử dụng các biến môi trường với tiền tố `P2P_`.

Một số biến môi trường thường sử dụng gồm:

- `P2P_PEER_ID`
- `P2P_BOOTSTRAP_HOST`

Cách tiếp cận này giúp hệ thống dễ dàng thay đổi thông số khi triển khai trên nhiều môi trường khác nhau mà không cần chỉnh sửa trực tiếp mã nguồn.

4.2.6. Kiểm thử hệ thống

Hệ thống được kiểm thử bằng framework `pytest` sau khi cài đặt các thư viện trong `requirements-dev.txt`. Các bài kiểm thử tập trung đánh giá những chức năng quan trọng của hệ thống như peer discovery, gửi và nhận tin nhắn, retry mechanism, synchronization và xử lý ACK.

4.3. Các module hệ thống

Hệ thống được thiết kế theo kiến trúc module nhằm tăng khả năng mở rộng, dễ bảo trì và thuận tiện cho quá trình phát triển cũng như kiểm thử. Mỗi module đảm nhiệm một chức năng riêng trong hệ thống P2P Chat.

4.3.1. Bootstrap Server

Bootstrap Server được sử dụng để hỗ trợ peer discovery trong hệ thống P2P Chat. Thành phần này dùng để tiếp nhận yêu cầu REGISTER, lưu thông tin peer vào registry và trả về `PEER_LIST` cho các peer mới tham gia mạng. Ngoài ra, Bootstrap Server còn dùng

để theo dõi heartbeat và loại bỏ các peer offline dựa trên last_seen và heartbeat_timeout. Các chức năng này được triển khai trong các file server.py và peer_registry.py.

4.3.2. Peer Manager

Peer Manager được sử dụng để quản lý lifecycle của peer và các kết nối TCP trong hệ thống. Thành phần này dùng để quản lý PeerList, lưu writer connection, giới hạn số lượng kết nối đồng thời (connect_limit) và xử lý kết nối tới các peer khác sau khi nhận PEER_LIST từ Bootstrap Server. Chức năng chính được triển khai trong node.py và peer_list.py.

4.3.3. Message Handler

Message Handler được sử dụng để xử lý toàn bộ quá trình tạo, gửi và nhận dữ liệu giữa các peer. Thành phần này dùng để tạo message thông qua new_message(), mã hóa và giải mã dữ liệu bằng encrypt_text() và decrypt_text(), xử lý ACK, synchronization và lưu lịch sử chat thông qua HistoryStore.add_message(). Hệ thống cũng sử dụng cơ chế deduplication với message_id và TTL 300 giây để tránh xử lý trùng lặp message. Các chức năng được triển khai trong message.py, crypto.py và history_store.py.

4.3.4. Connection Handler

Connection Handler được sử dụng để quản lý toàn bộ kết nối TCP giữa các peer trong mạng. Thành phần này dùng để mở server socket bằng asyncio.start_server(), thiết lập outbound connection bằng open_peer_connection() và xử lý đọc/ghi dữ liệu thông qua line-delimited JSON message. Các chức năng được triển khai trong server.py và client.py.

4.3.5. Heartbeat Manager

Heartbeat Manager được sử dụng để duy trì trạng thái hoạt động của các peer trong mạng. Thành phần này dùng để gửi heartbeat định kỳ tới Bootstrap Server và các peer đang kết nối nhằm cập nhật last_seen và phát hiện peer offline. Nếu vượt quá heartbeat_timeout, peer sẽ bị loại khỏi PeerList. Chức năng được triển khai trong file heartbeat.py.

4.3.6. Event Hub

Event Hub được sử dụng để hỗ trợ cơ chế realtime cho hệ thống. Thành phần này dùng để publish các sự kiện như message_received, message_sent, peer_joined và peer_left tới các subscriber thông qua queue bất đồng bộ. Cơ chế publish-subscribe này giúp WebSocket và giao diện frontend cập nhật dữ liệu realtime mà không cần reload trang. Chức năng được triển khai trong file events.py.

4.3.7. Web Server

Web Server được sử dụng để cung cấp REST API, WebSocket realtime và giao diện web cho người dùng. Thành phần này dùng để hiển thị dữ liệu chat, trạng thái peer và cập nhật thông tin theo thời gian thực thông qua WebSocket. Các chức năng được triển khai trong `web_server.py` và thư mục web chứa frontend của hệ thống.

4.3.8. CLI

CLI được sử dụng để người dùng tương tác trực tiếp với hệ thống thông qua terminal. Thành phần này dùng để gửi tin nhắn, xem danh sách peer, theo dõi metrics và kiểm tra trạng thái hoạt động của hệ thống. CLI được xây dựng bằng `prompt-toolkit` và `typer` nhằm hỗ trợ giao diện dòng lệnh linh hoạt và thuận tiện hơn cho quá trình sử dụng.

CHƯƠNG 5: THỬ NGHIỆM VÀ ĐÁNH GIÁ

5.1. Kịch bản thử nghiệm

Để đánh giá khả năng hoạt động của hệ thống P2P Chat, nhóm thực hiện xây dựng nhiều kịch bản thử nghiệm khác nhau nhằm kiểm tra các chức năng chính, khả năng chịu lỗi và mức độ ổn định của hệ thống trong môi trường mạng phân tán.

Các thử nghiệm được thực hiện trên nhiều peer chạy đồng thời trong mạng LAN và môi trường local.

5.1.1. Thử nghiệm nhiều peer kết nối đồng thời

Trong kịch bản này, nhiều peer được khởi động cùng lúc và thực hiện đăng ký với Bootstrap Server để tham gia mạng.

Quy trình thử nghiệm:

1. Khởi động Bootstrap Server.
2. Khởi động nhiều Peer Node trên các terminal khác nhau.
3. Các peer gửi yêu cầu REGISTER tới Bootstrap Server.
4. Bootstrap Server trả về danh sách peer đang hoạt động.
5. Các peer thiết lập kết nối TCP trực tiếp với nhau.

Kết quả cho thấy:

- Các peer có thể tham gia mạng thành công.
- Peer discovery hoạt động ổn định.
- Danh sách peer được cập nhật chính xác.
- Hệ thống duy trì được nhiều kết nối TCP đồng thời.

5.1.2. Thử nghiệm gửi tin nhắn đồng thời

Kịch bản này kiểm tra khả năng xử lý nhiều tin nhắn được gửi cùng lúc giữa các peer.

Quy trình thực hiện:

1. Khởi động nhiều peer trong mạng.
2. Các peer gửi tin nhắn đồng thời đến nhiều peer khác nhau.
3. Hệ thống thực hiện:

- mã hóa dữ liệu,
- gửi message,
- xử lý ACK,
- lưu history.

Kết quả thử nghiệm:

- Tin nhắn được truyền thành công giữa các peer.
- ACK được xử lý đúng.
- Message deduplication hoạt động ổn định.
- WebSocket UI cập nhật dữ liệu theo thời gian thực.

Trong một số trường hợp tải cao, độ trễ tăng nhẹ do SQLite lock và số lượng kết nối TCP đồng thời.

5.1.3. Thử nghiệm peer rời mạng

Mục tiêu của thử nghiệm này là kiểm tra khả năng cập nhật trạng thái peer khi một node rời khỏi hệ thống.

Quy trình:

1. Peer đang hoạt động bình thường trong mạng.
2. Một peer chủ động đóng ứng dụng hoặc gửi LEAVE.
3. Các peer khác và Bootstrap Server kiểm tra heartbeat timeout.

Kết quả:

- Peer offline được loại khỏi PeerList.
- Các kết nối TCP liên quan được đóng.
- Danh sách peer online được cập nhật chính xác.
- Hệ thống tiếp tục hoạt động bình thường với các peer còn lại.

5.1.4. Thử nghiệm mất kết nối

Kịch bản này mô phỏng tình huống mất mạng hoặc mất kết nối đột ngột giữa các peer.

Quy trình:

1. Thiết lập kết nối giữa nhiều peer.
2. Ngắt mạng hoặc tắt peer đột ngột.
3. Quan sát cơ chế retry và heartbeat.

Kết quả:

- Hệ thống phát hiện timeout thông qua heartbeat.
- Cơ chế retry được kích hoạt tối đa 3 lần.
- Message gửi thất bại được đánh dấu tương ứng.
- Các peer còn lại vẫn duy trì hoạt động.

Tuy nhiên, do hệ thống chưa hỗ trợ persistent outgoing queue nên các tin nhắn thất bại chưa được retry lâu dài sau khi process restart.

5.2. Kết quả thử nghiệm

Qua quá trình thử nghiệm, hệ thống đã thực hiện thành công các chức năng chính:

- peer discovery,
- direct chat,
- group chat,
- heartbeat,
- synchronization,
- ACK-based delivery,
- realtime WebSocket update.

Hệ thống hoạt động ổn định trong môi trường có nhiều peer kết nối đồng thời và vẫn duy trì được khả năng giao tiếp khi một số peer gặp lỗi hoặc rời mạng.

Các cơ chế như:

- retry,
- deduplication,
- heartbeat timeout

đã hỗ trợ cải thiện tính ổn định và khả năng chịu lỗi của hệ thống.

5.3. Đánh giá hiệu năng

5.3.1. Độ trễ truyền tin

Trong môi trường mạng LAN, thời gian truyền tin giữa các peer tương đối thấp và gần như realtime.

Độ trễ chủ yếu phụ thuộc vào:

- số lượng peer đang kết nối,
- tốc độ mạng,
- số lượng message xử lý đồng thời,
- tốc độ ghi dữ liệu SQLite.

Khi số lượng kết nối tăng cao, thời gian xử lý ACK và ghi dữ liệu có thể tăng nhẹ.

5.3.2. Độ ổn định hệ thống

Hệ thống duy trì hoạt động ổn định trong quá trình thử nghiệm:

- nhiều peer kết nối đồng thời,
- gửi tin nhắn liên tục,
- peer rời mạng,
- mất kết nối đột ngột.

Các cơ chế heartbeat và retry giúp hệ thống phát hiện lỗi và tự động cập nhật trạng thái peer.

5.3.3. Khả năng mở rộng

Hệ thống có khả năng mở rộng ở mức cơ bản nhờ mô hình phân tán Peer-to-Peer.

Việc truyền dữ liệu trực tiếp giữa các peer giúp giảm tải cho Bootstrap Server. Tuy nhiên, hiệu năng vẫn bị ảnh hưởng bởi:

- số lượng kết nối TCP đồng thời,
- giới hạn SQLite single writer,
- chi phí duy trì heartbeat và synchronization.

Đối với quy mô lớn hơn, hệ thống cần:

- connection pooling,
- tối ưu database,
- cơ chế distributed discovery hoàn chỉnh.

5.4. Đánh giá cơ chế xử lý lỗi

Hệ thống đã triển khai nhiều cơ chế xử lý lỗi nhằm tăng tính ổn định trong môi trường mạng phân tán.

a. Heartbeat và timeout

Heartbeat được sử dụng để phát hiện peer offline. Nếu peer không phản hồi trong khoảng thời gian quy định, peer sẽ bị loại khỏi danh sách kết nối.

b. Retry mechanism

Khi gửi message thất bại, hệ thống thực hiện retry tối đa 3 lần trước khi đánh dấu message failed.

Cơ chế này giúp tăng xác suất truyền tin thành công trong trường hợp lỗi mạng tạm thời.

c. Deduplication

Mỗi message được gán message_id duy nhất.

Hệ thống lưu cache message_id trong khoảng thời gian TTL nhằm tránh xử lý trùng lặp message khi retry hoặc reconnect.

d. Graceful degradation

Trong trường hợp Bootstrap Server không hoạt động:

- các peer đang kết nối vẫn có thể tiếp tục chat,
- peer discovery vẫn có thể duy trì thông qua seed peers hoặc PEER_EXCHANGE.

Điều này giúp giảm phụ thuộc hoàn toàn vào Bootstrap Server.

5.5. Hạn chế hệ thống

Mặc dù hệ thống đã triển khai được nhiều chức năng cơ bản của mạng P2P, vẫn còn tồn tại một số hạn chế như:

- Chưa hỗ trợ end-to-end encryption theo chuẩn hiện đại.

- Chưa có cơ chế xác thực peer mạnh.
- Chưa hỗ trợ NAT traversal nâng cao.
- Chưa có persistent outgoing queue cho offline delivery.
- SQLite chưa phù hợp với quy mô lớn.
- Bootstrap Server vẫn là điểm phụ thuộc trong quá trình discovery.
- Hệ thống chưa đảm bảo delivery khi recipient offline.
- Chưa có causal ordering hoặc global message ordering.

Ngoài ra, cơ chế mã hóa hiện tại sử dụng XOR stream cipher nên mức độ bảo mật chưa cao so với các chuẩn hiện đại như AES-GCM

Trong tương lai, hệ thống cần được cải tiến thêm về:

- bảo mật,
- khả năng mở rộng,
- reliable delivery,
- distributed discovery,
- và cơ chế đồng bộ dữ liệu nâng cao.

CHƯƠNG 6: KẾT QUẢ HỆ THỐNG

6.1. Kết quả xây dựng hệ thống

Sau quá trình nghiên cứu và triển khai, hệ thống P2P Chat đã xây dựng thành công mô hình chat ngang hàng hoạt động theo kiến trúc Peer-to-Peer. Hệ thống cho phép nhiều peer tham gia mạng, thiết lập kết nối trực tiếp và trao đổi dữ liệu mà không phụ thuộc hoàn toàn vào server trung tâm.

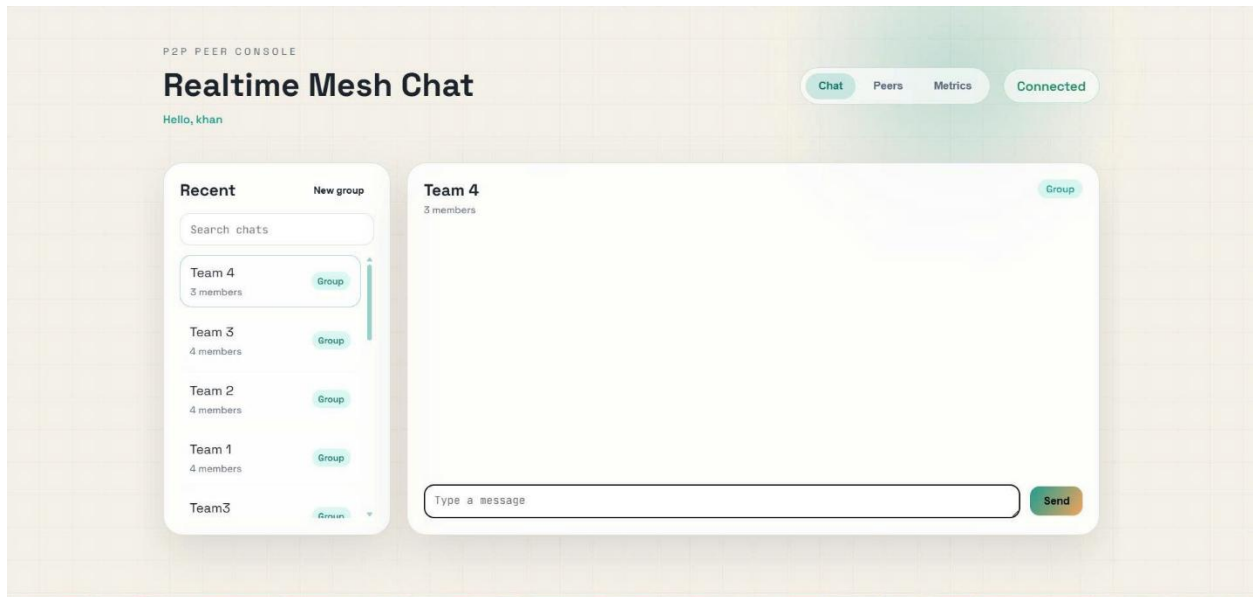
Phiên bản hiện tại đã triển khai được các chức năng chính bao gồm:

- peer discovery thông qua Bootstrap Server
- gửi và nhận tin nhắn trực tiếp giữa các peer
- hỗ trợ chat nhóm
- broadcast thông báo đến tất cả các peer
- quản lý trạng thái online/offline
- cơ chế heartbeat kiểm tra kết nối
- ACK và retry cho truyền tin
- lưu trữ lịch sử chat bằng SQLite
- giao diện realtime sử dụng FastAPI và WebSocket.

Ngoài ra, hệ thống còn hỗ trợ cơ chế mã hóa cơ bản cho dữ liệu truyền tải giữa các peer nhằm tăng tính bảo mật trong quá trình giao tiếp.

6.2. Giao diện và hoạt động của hệ thống

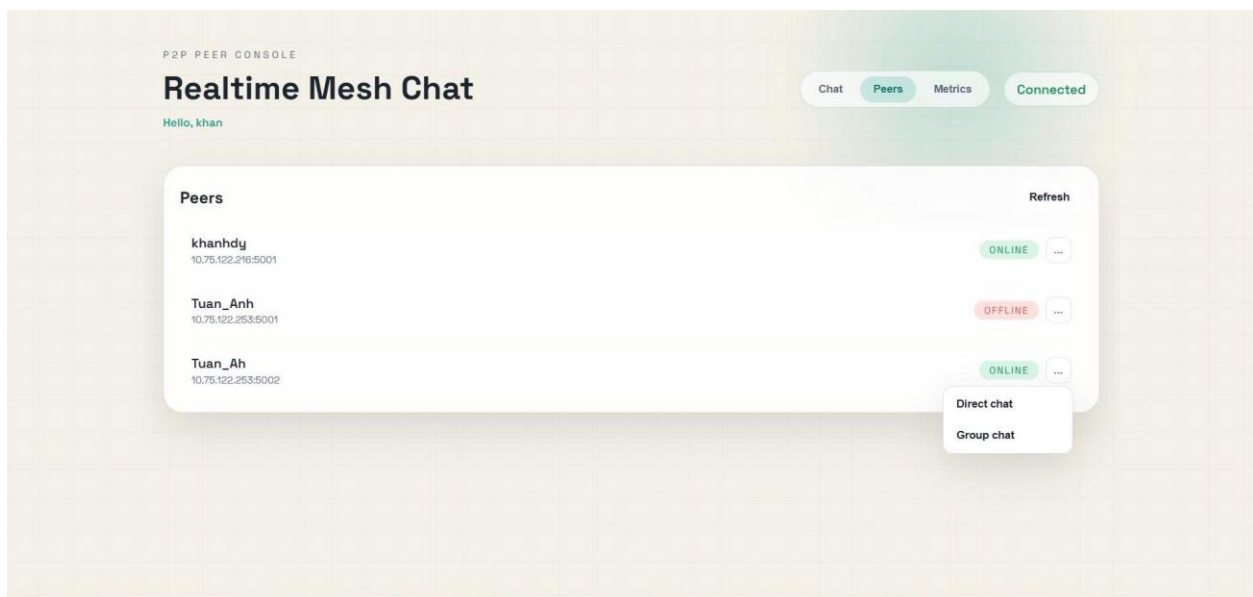
6.2.1 giao diện chat



Hình 3 Giao diện chat

Giao diện chat bao gồm các peer và group từng chat gần đây. Để bắt đầu chat, người dùng chọn peer hoặc group mình muốn chat cùng và bắt đầu chat.

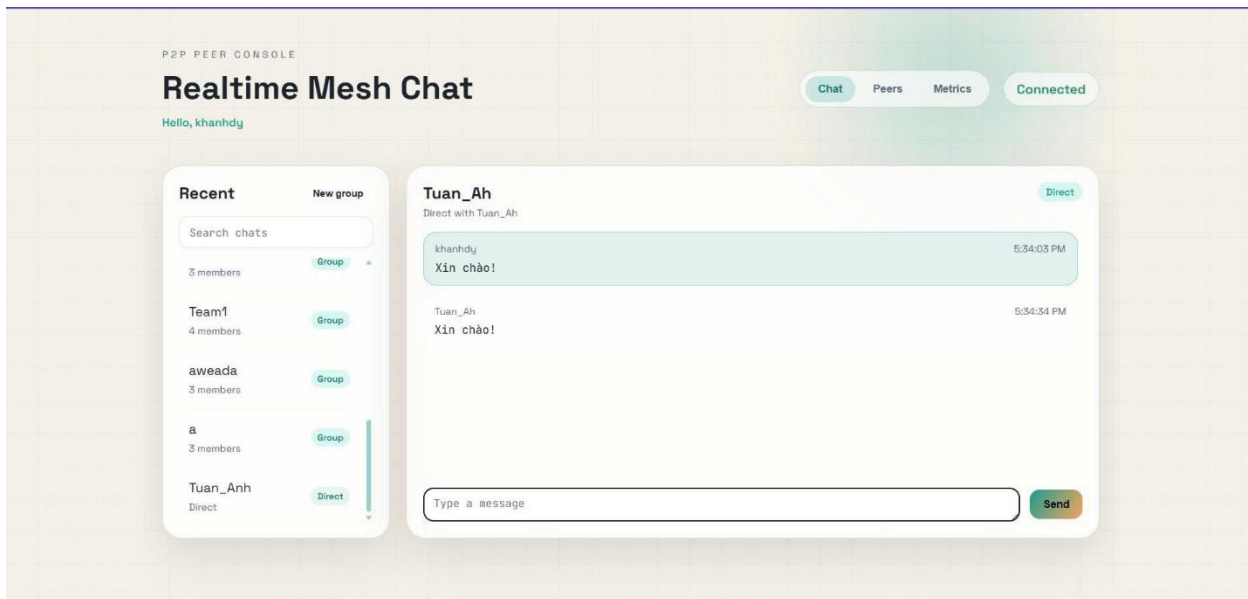
6.2.2 Giao diện người dùng (peer)



Hình 4 Giao diện người dùng (peer)

Giao diện người dùng hiển thị các peer và trạng thái hoạt động (Online, Offline). Người dùng có thể chọn nút ba chấm để thực hiện thao tác chat trực tiếp hoặc tạo group chat.

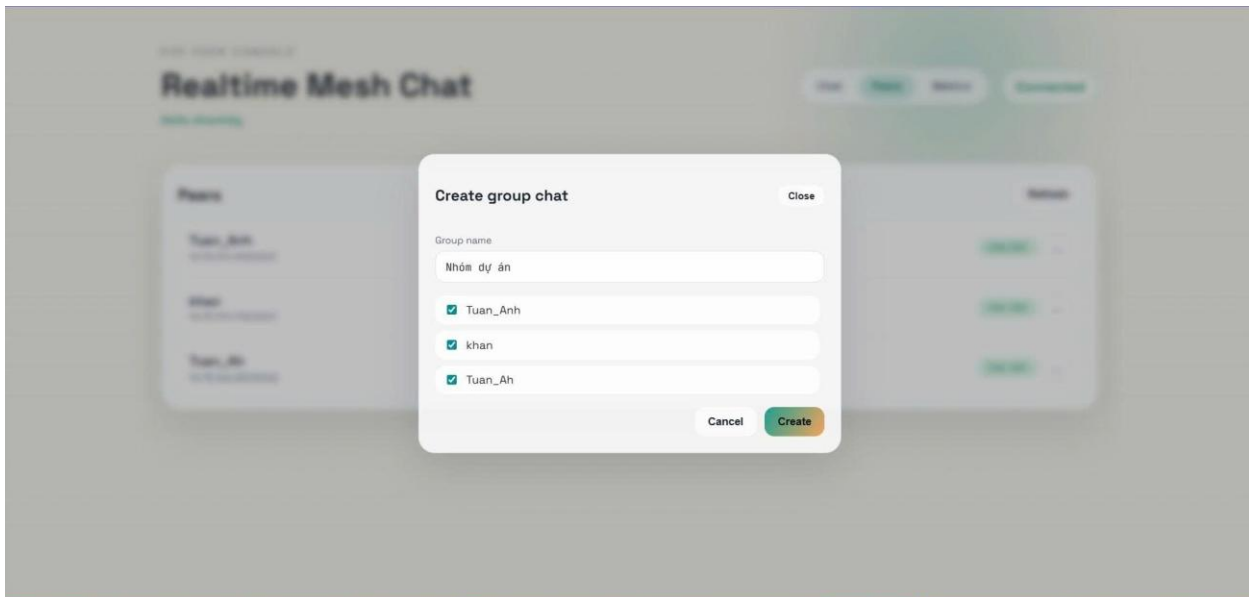
Trường hợp 1 : chat cá nhân 1-1 (kết quả sau khi ấn Direct chat)



Hình 5 Giao diện chat 1-1

Sau khi chọn Direct chat, người dùng sẽ được đưa tới trang chat với peer đã chọn trước đó.

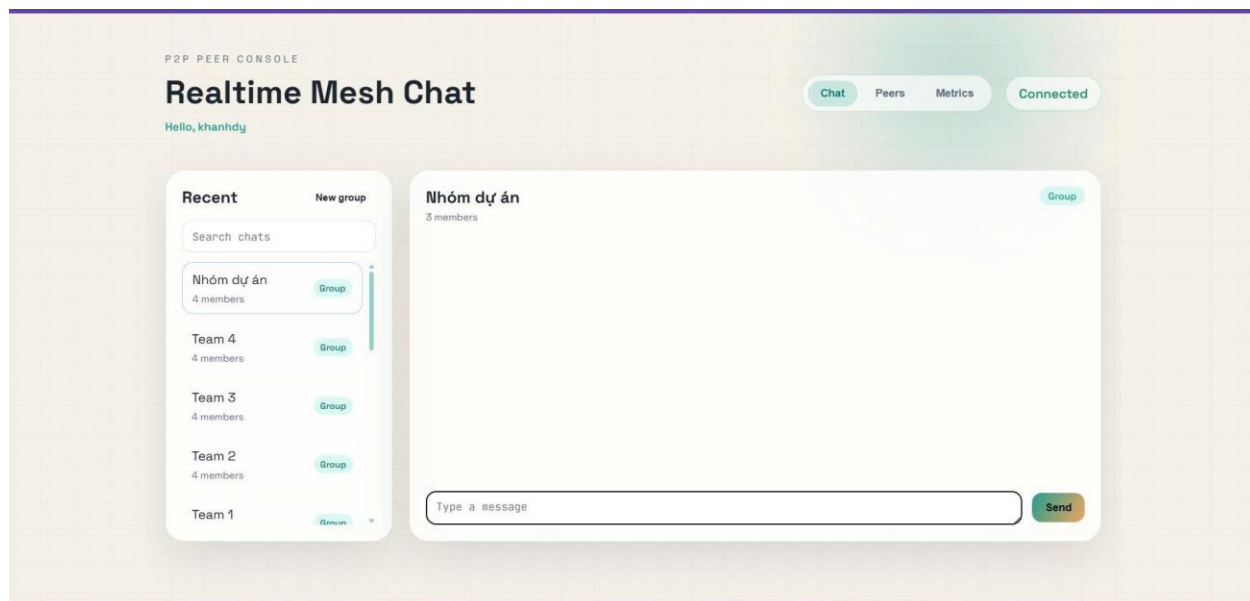
Trường hợp 2 : Tạo Group Chat (kết quả sau khi ấn Group chat)



Hình 6 Giao diện tạo group chat

Sau khi chọn group chat, người dùng có thể đặt tên nhóm và lựa chọn những người muốn thêm vào group.

Kết quả sau khi nhấn create



Hình 7 Giao diện group chat

Sau khi bấm chọn tạo group, người dùng được đưa đến trang chat của group đã tạo và có thể chat với những peer đã chọn để lập group.